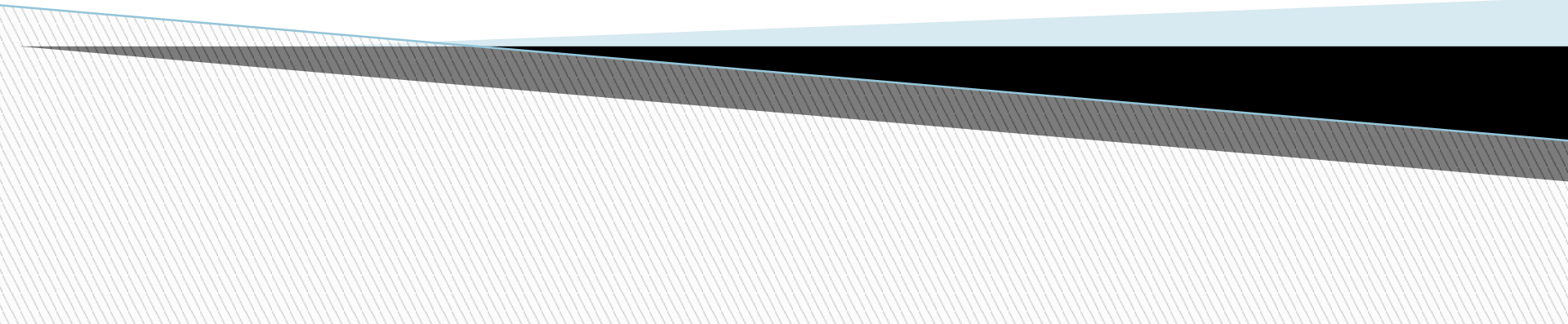


Complete Binary Tree

Data Structure

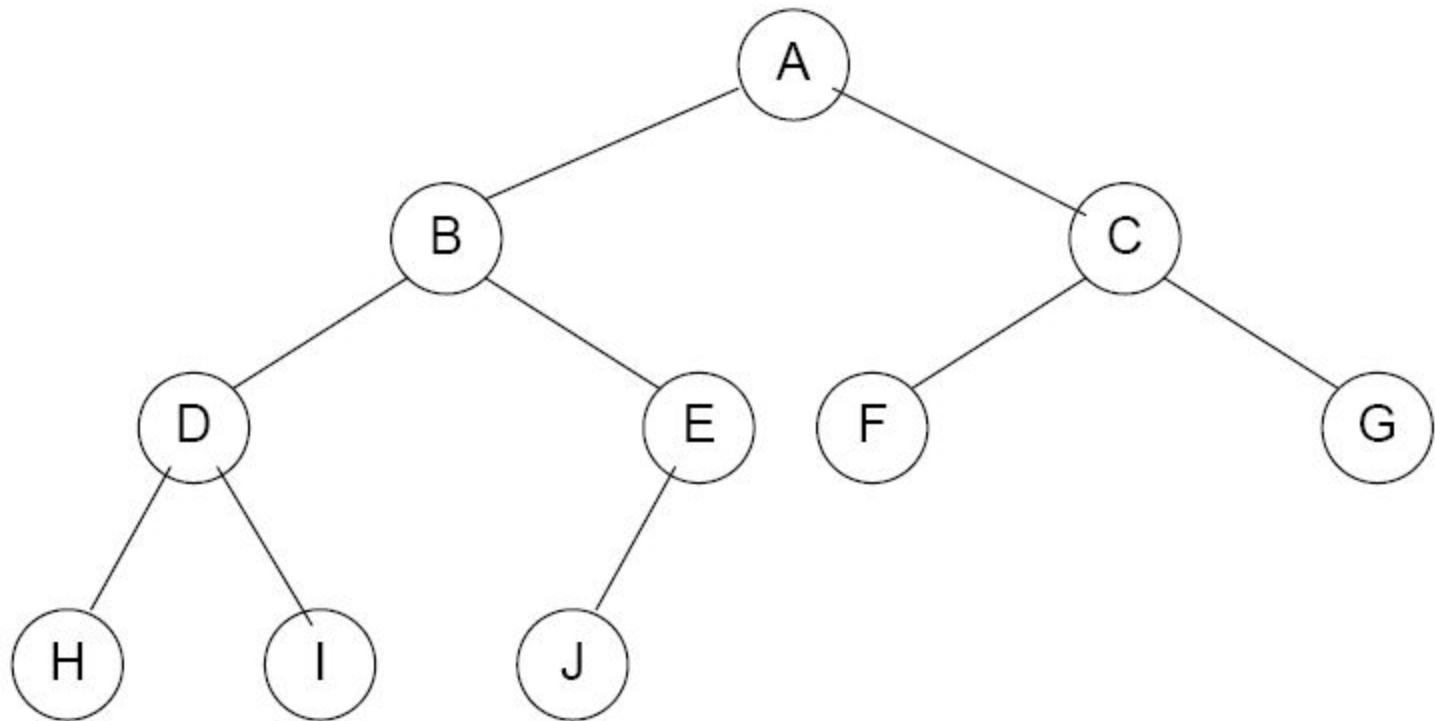


Complete Binary Tree

Definition:

- A complete binary tree is a tree that is completely filled, with the possible exception of the bottom level.
- The bottom level is filled from left to right.

Complete Binary Tree

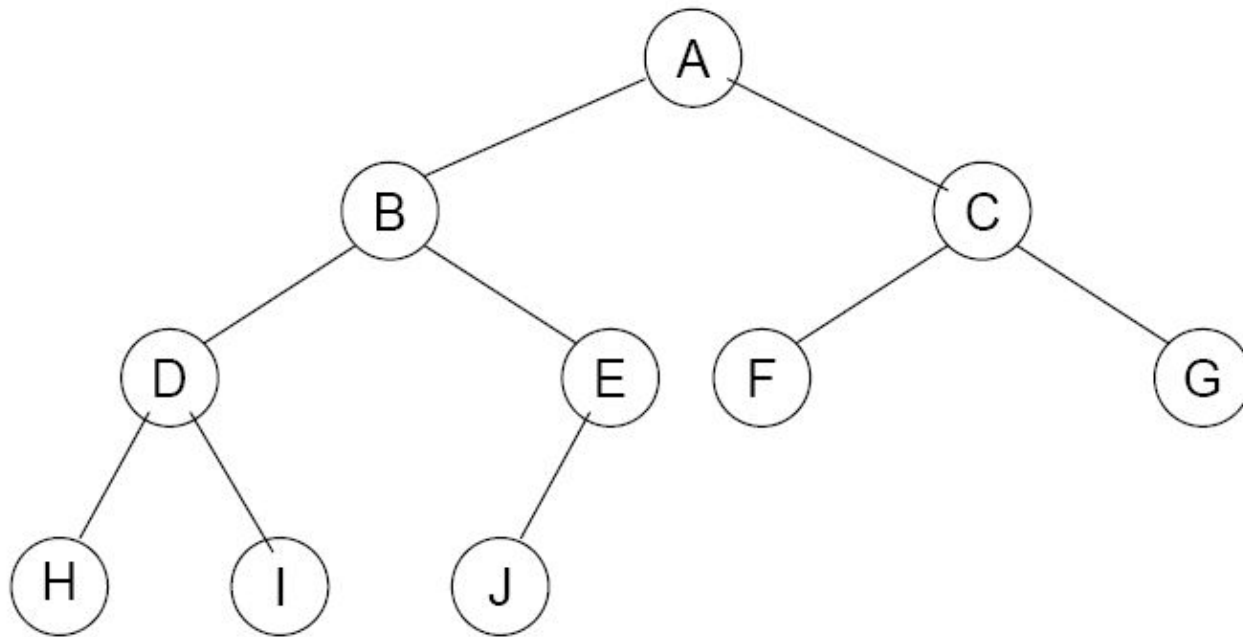


A complete binary tree of height h has between 2^h and $2^{h+1} - 1$ nodes.

Complete Binary Tree

- A complete binary tree of height h has between 2^h and $2^{h+1} - 1$ nodes.
- This implies that the height of a complete binary tree is $\lceil \log N \rceil$, which is $O(\log N)$.
- **Because the tree is so regular, it can be stored in an array. No pointers are necessary.**

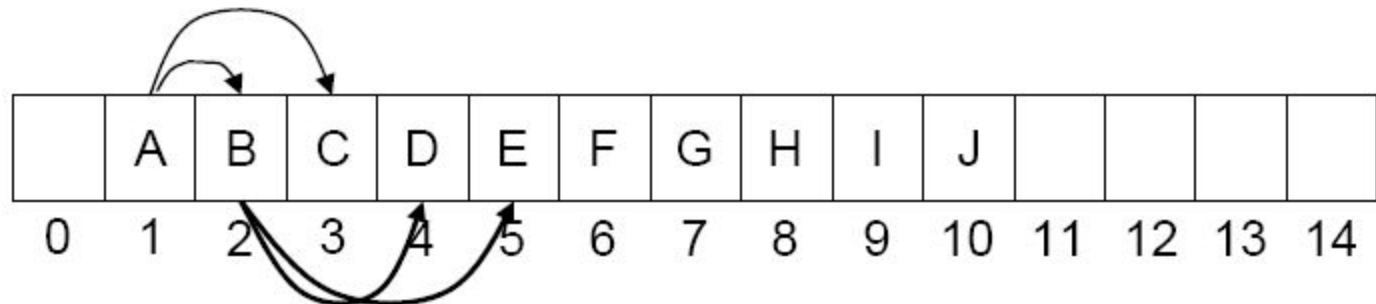
Example



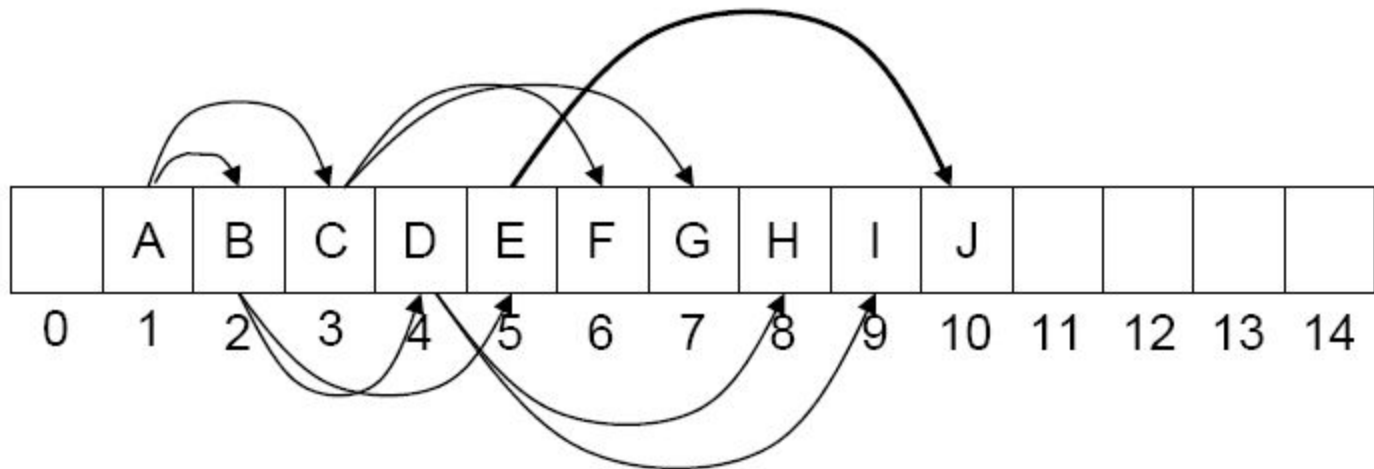
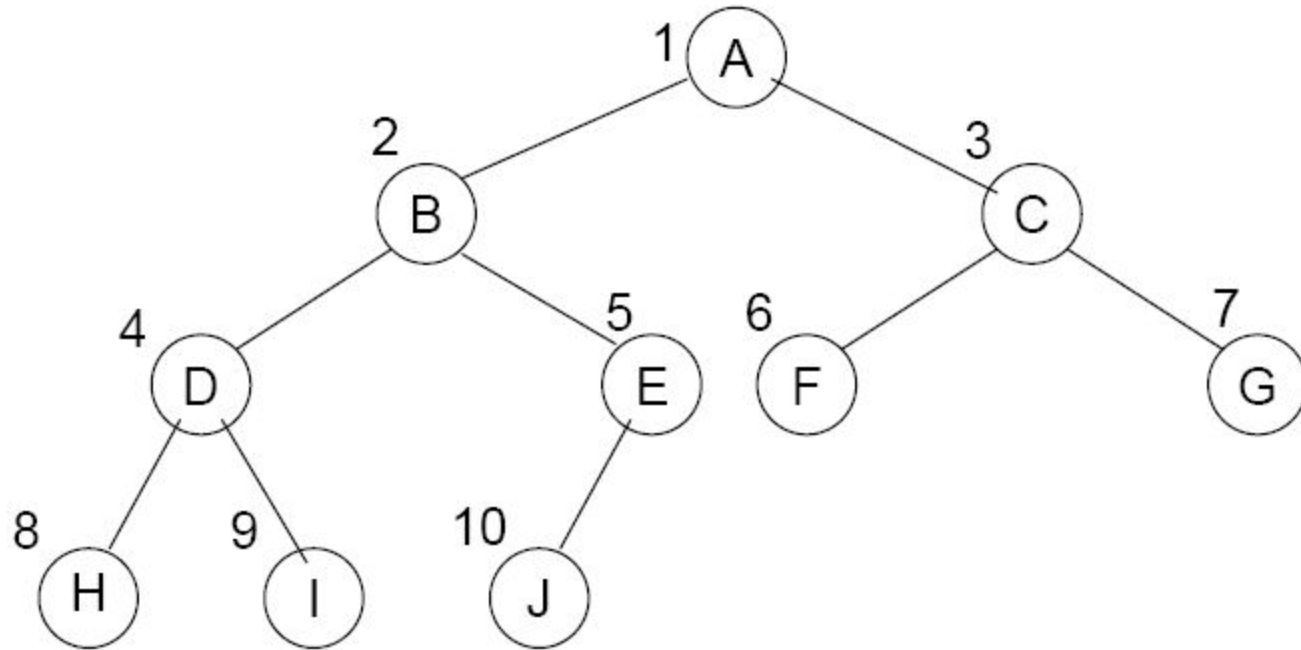
	A	B	C	D	E	F	G	H	I	J				
	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Rule:

- For any array element at position i , the left child is at $2i$, the right child is at $(2i + 1)$ and the parent is at $\text{floor}(i/2)$.
- Explanation.**
 - Take the 2nd element i.e. B , it has two children D and E . The position of B is 2 i.e. the value of i is 2. Its left child D will be at position $2i$ i.e. 4th position and its right child E will be at position $2i+1$ i.e. 5.



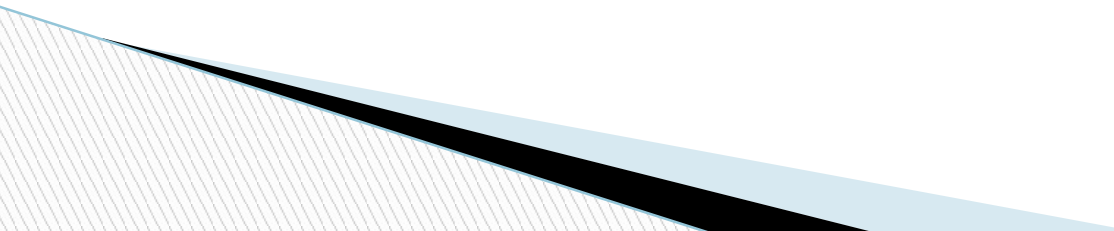
Binary Tree in Array.



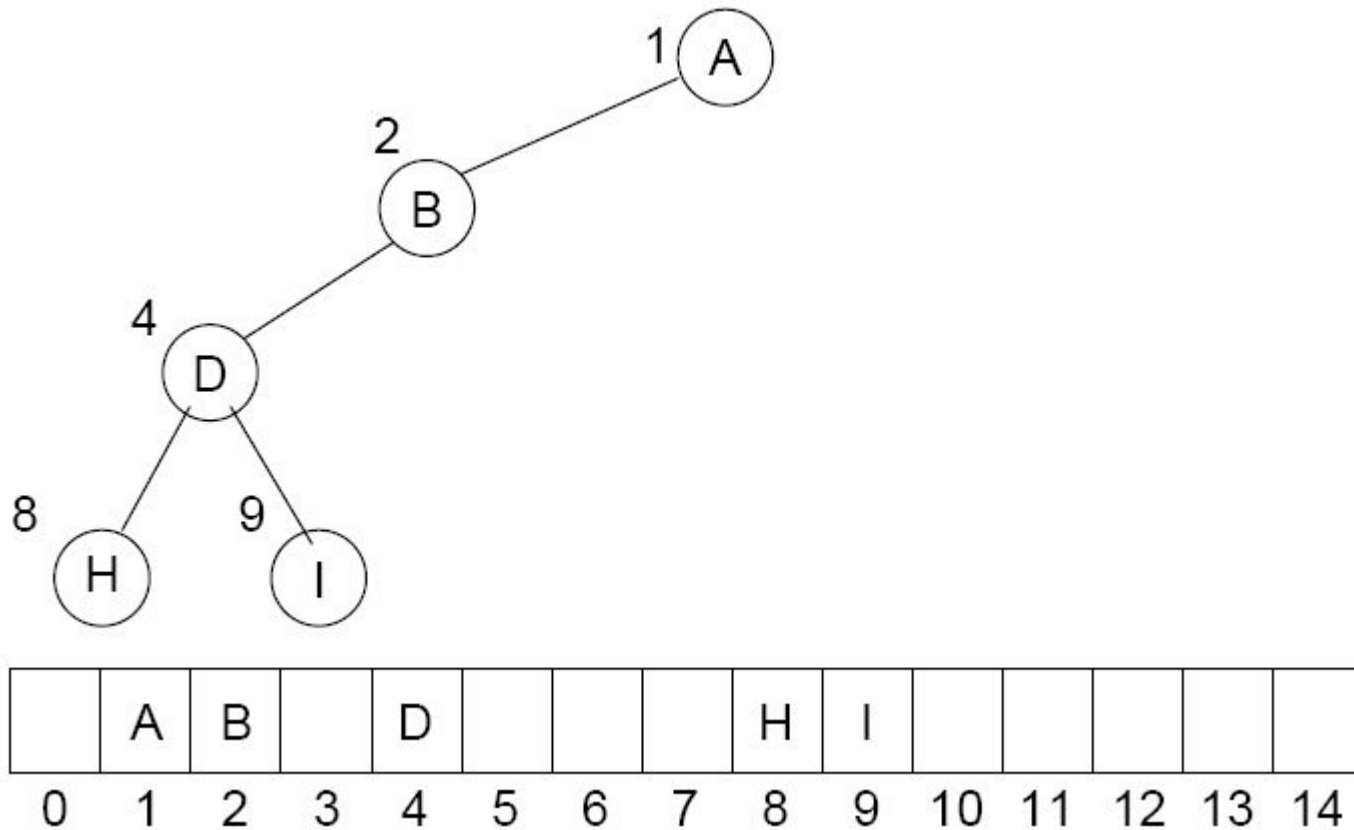
Array Vs Pointers

- The array is a very fast and efficient data structure and is supported by the compiler.
- Use of pointers is beneficial in some situation.
- Example: Balancing of AVL tree
 - Here pointers are more efficient.
 - If we are using array, there will be need of moving a large data here and there to balance the tree.

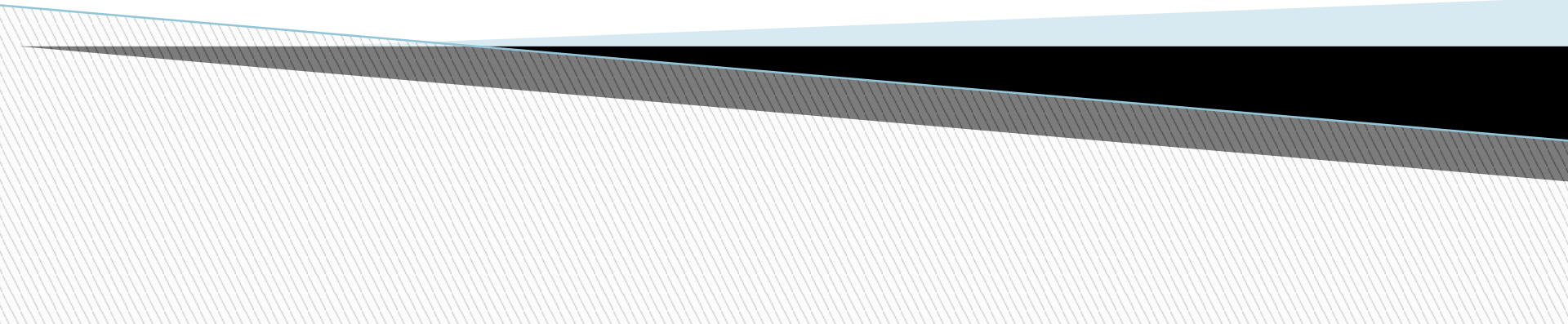
Why can't we use an array for a Binary Tree

- ❑ Arrays are fixed size data structure. If array is full and you want to insert one more element you will need to create a new array of larger size and copy previous array. It is a lot of overhead.
 - ❑ If you create a very large array in advance you will waste memory.
 - ❑ For a sparse tree most of array positions will be wasted
- 

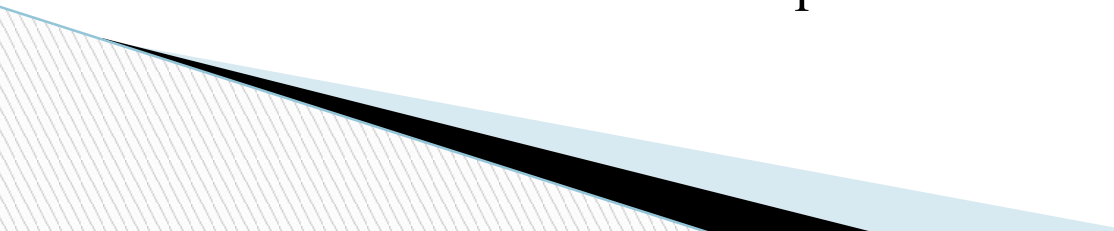
Not a complete Binary Tree



Heap



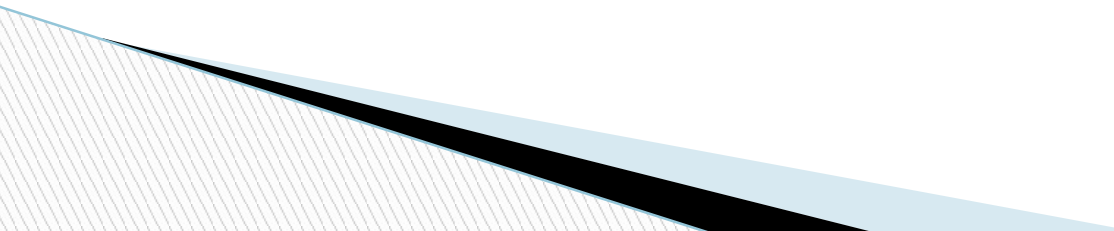
Priority queue

- Heap is a data structure of big use and benefit.
 - It is used in **priority queue**(special kind of queue).
 - This priority queue does not follow the FIFO rule.
 - We put the elements in the queue at the end but later got the element with respect to its priority.
-
- **Example:**
 - one job might be particularly important, so it might be desirable to allow that job to be run as soon as the printer is available.
 - In a multiuser environment, the operating system scheduler must decide which of several processes to run
- 

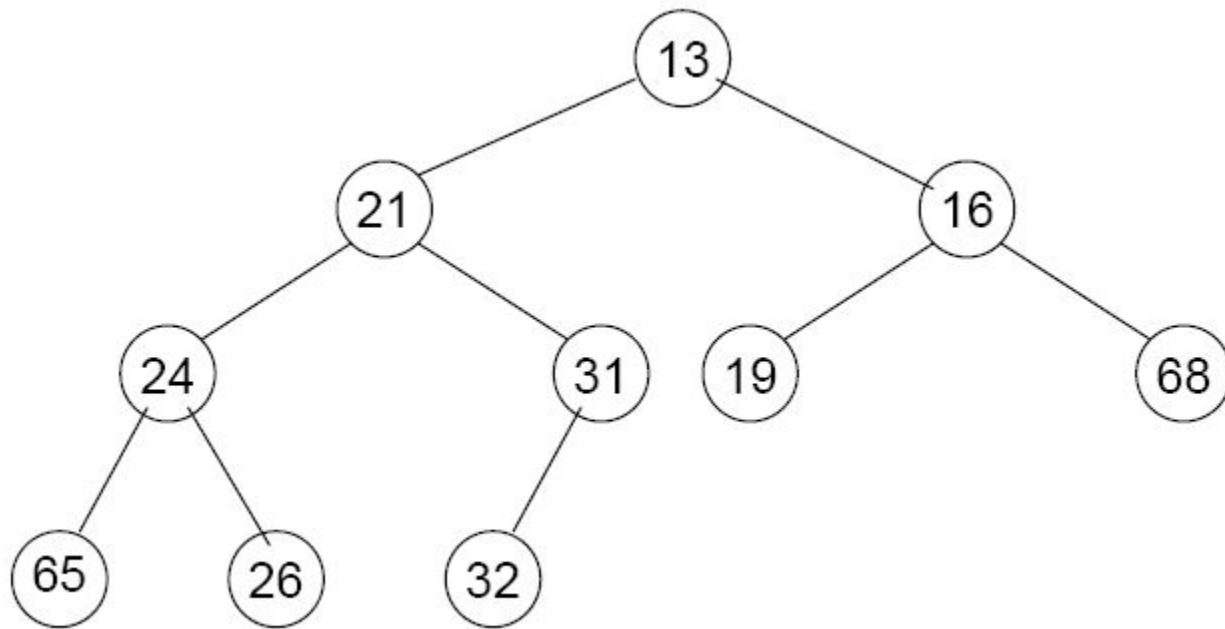
Heap

- Thus the implementation of priority queue with an array is not efficient in terms of time. There is an alternate that is called heap.
- **Major uses** in priority queue and other is in sorting etc.

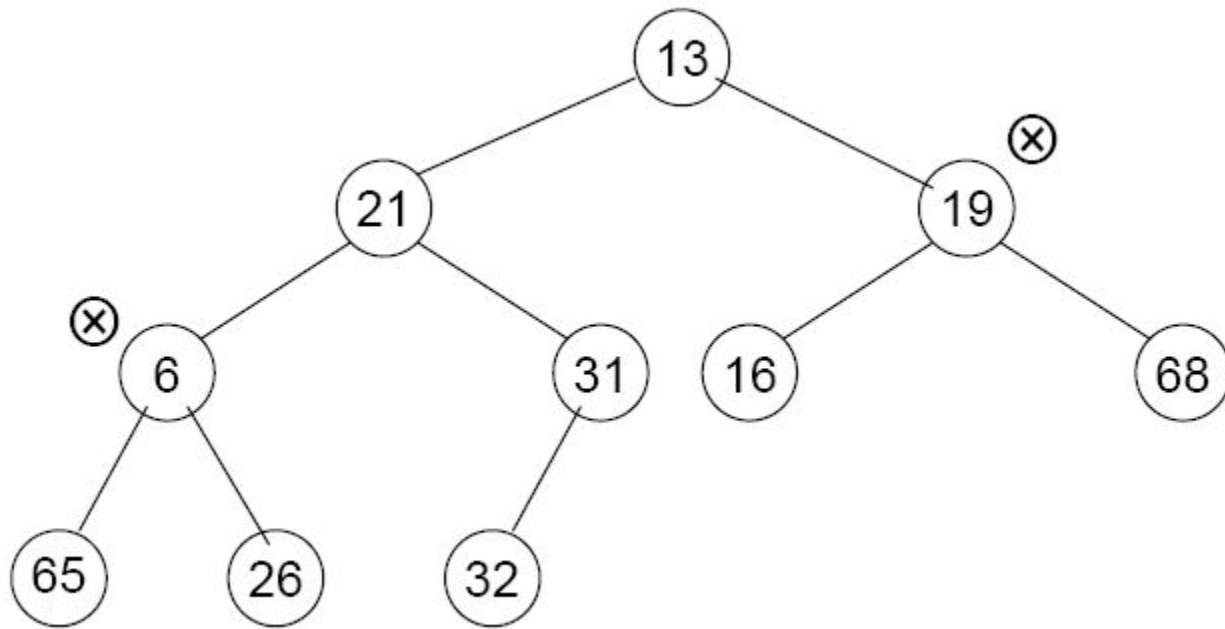
Heap

- The **definition of heap** is that it is a complete binary tree that conforms to the **heap order**.
 - The **heap order** is a property that states that in a (min) heap for every node X , the key in the parent is smaller than (or equal to) the key in X .
 - In other words, the parent node has key(value) smaller than or equal to both of its children nodes.
- 

Example: Min Heap



Neither a search tree, nor a heap



Max Heap

